

Задача А. Новый язык

Это задача на реализацию. Основными идейными сложностями являются корректное выполнение операций по модулю и обработка некорректных программ.

Различие между задачами состояло в функциях, необходимых для реализации, и количестве аргументов у функций `add` и `sub`.

Основной идеей было исполнение и проверка на корректность построчно. Если мы в какой-то момент встречаем некорректную строку, то после неё сразу завершаемся (делая `exit(0)`).

При написании на языке C++ можно использовать тип данных `unsigned int`, тогда все операции, кроме умножения, будут эквивалентны обычным операциям с этим типом данных. При написании на языке Python необходимо после каждой операции брать остаток по модулю 2^{32} .

Для умножения можно было использовать тип данных `unsigned long long` в C++. Тогда после перемножения двух чисел, старшую и младшую часть можно распределить взятием по модулю и делением на 2^{32} . Для Python необходимо было просто перемножить два числа и правильно их записать в регистры.

При встрече команды `print` необходимо было сразу выводить значение регистра, который был указан как аргумент. Тогда при встрече некорректной строки все `print` выведут свои значения, что и требуется по условию.

Теперь рассмотрим, как можно было обрабатывать некорректные строки. Будем считывать в следующем порядке:

1. Считываем первое слово (в Python считаем строку, разделим ее и посмотрим на первое слово), если слово не является одной из доступных команд, то выводим номер строки и завершаемся;
2. Если же название команды корректно, то гарантируется корректность количества аргументов. Тогда необходимо проверить только имена регистров. Для этого можно было создать `std::set` с допустимыми регистрами и проверять, есть ли в нем такое имя.

Также для уменьшения количества кода каждую функцию можно было выразить через себя же с большим количеством аргументов, тогда проверка регистров была бы только в функции с максимальным количеством аргументов.

Так как при корректности имени команды некорректным могло быть только имя регистра, то данный подход рассматривает все возможные случаи.

Тогда итоговое время работы будет $O(n)$ и затраченная память $O(1)$.

Задача С. Лазерный лабиринт

Заметим, что отличие между задачами состояло только в типах зеркала в лабиринте. В коде это отличие отображается в обработке зеркала, что не влияет на сложность.

Одной из сложностей задачи является возможность попасть в цикл.

Основной идеей в данной задаче было `dfs` с сохранением состояний. Состоянием является тройка (x, y, dir) , где dir — одно из четырех направлений.

Из каждого состояния есть ровно один исход:

- После отражения и шага мы попадаем в новую клетку, тогда движение продолжается;
- Мы находимся в стене или за границами поля, тогда движение луча завершается.

Сначала покажем, как обрабатывать циклы.

Давайте для состояний заведем массив `dp[n][m][4]`, в котором будем хранить ответ для соответствующего запроса. Тогда при входе в состояние будем помечать, что текущее состояние находится в обработке. Если в какой-то момент мы придем в такое состояние, то это означает, что мы попали в цикл. Тогда все состояния, что мы обошли ранее в текущем обходе так же попадут в цикл и будут иметь ответ -1. Это так же можно сохранить в массиве `dp`.

Покажем теперь, как обрабатывать запрос, когда мы не попадаем в цикл.

В `dp` при попадании в стену давайте сохраним ответ как 0, а при выходе за границу сразу же будем возвращать 0.

Тогда $\text{dp}[x][y][dir]$ будет равно $\text{dp}[x'][y'][dir'] + 1$, где x' – новая позиция по x , y' – новая позиция по y , dir' – новое направление.

Итого у нас есть 4 различных состояния в массиве dp :

- Еще не начали обработку, тогда при заходе в такое состояние мы рекурсивно считаем для него ответ;
- Состояние находится в обработке, тогда при заходе в него мы запоминаем, что ответом будет -1;
- Состояние уже было посчитано и ответ в нем -1, тогда мы просто возвращаем -1, так как мы попадем в цикл.
- Состояние уже было посчитано и ответ в нем не -1, тогда мы так же возвращаем значение в текущем состоянии, так как луч завершит движение.

У нас получился обход dfs который сохраняет ответ для своего состояния, что работает быстро, а именно за $O(nm + q)$ по времени и $O(nm)$ по памяти, так как каждое состояние будет обработано не более 1 раза.